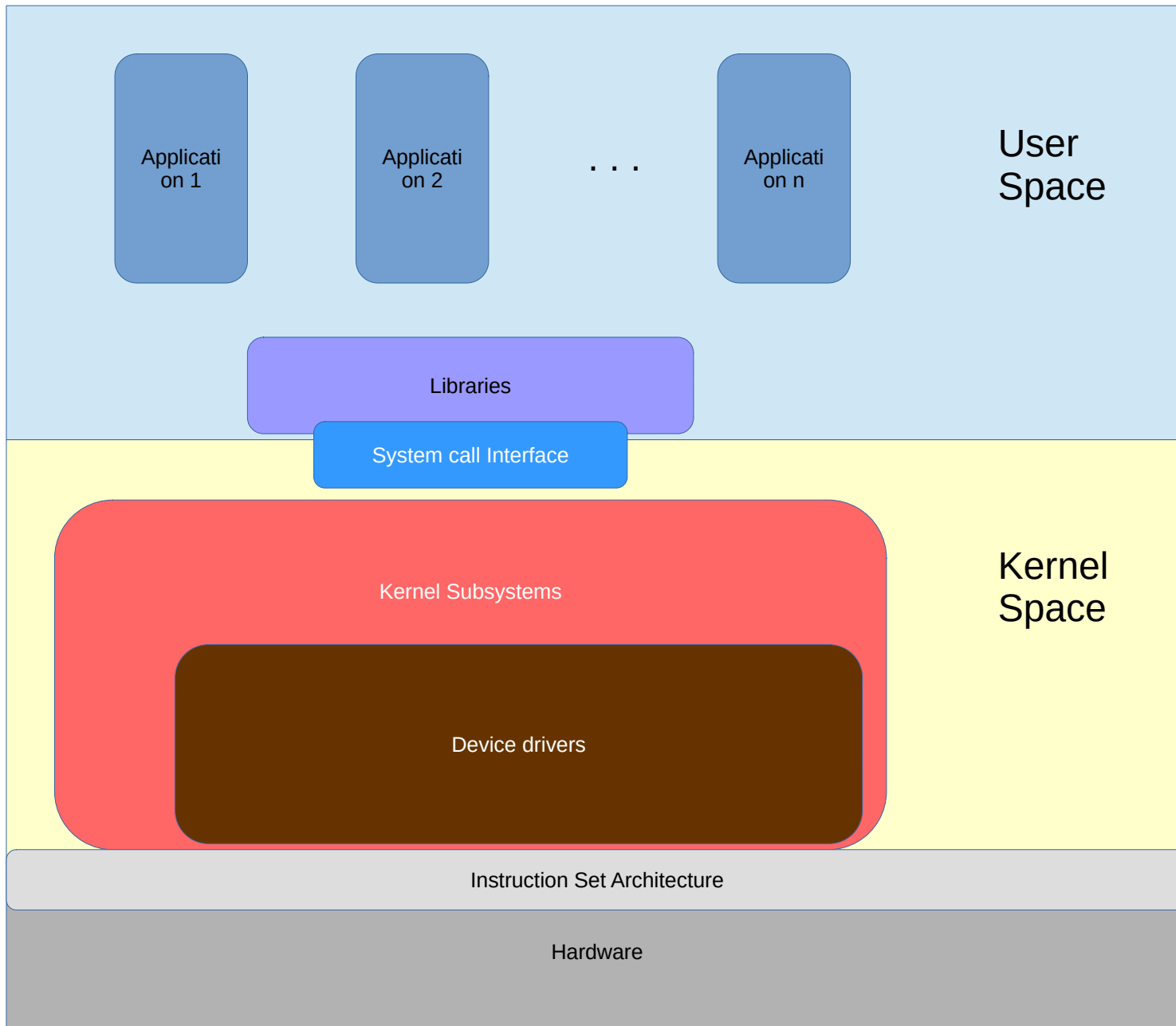


System Calls Intro

Linux Overall Architecture



Linux Kernel Overview

Kernel Classification

Monolithic Kernel

- Single binary
- Running in single address space
- Direct function invocation
- Eg: Unix

MicroKernel

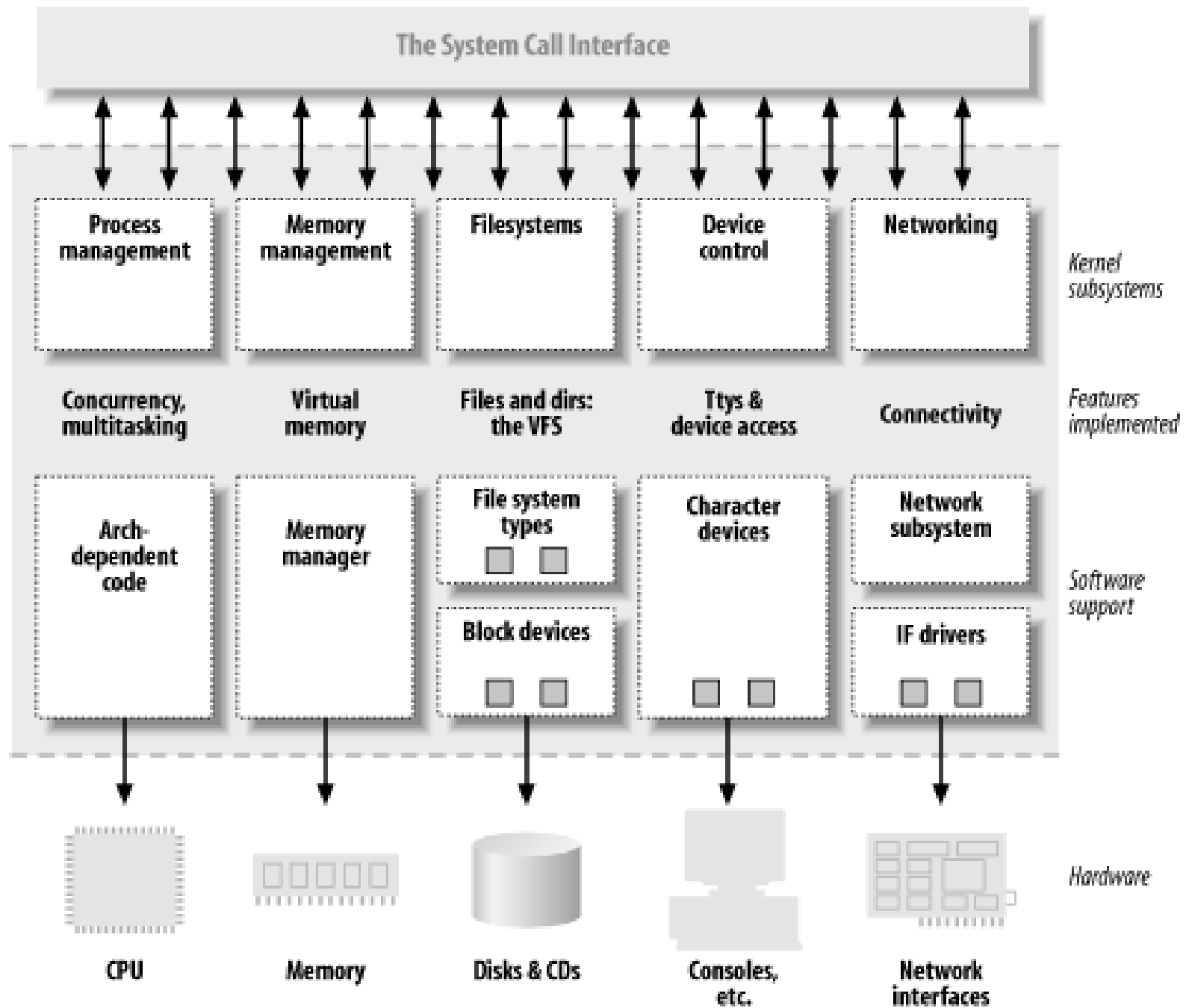
- Broken down into server processes
- Run in different address space
- Invocation by message passing
- Privilege execution is provided when absolutely necessary
- Eg: windows NT, Mach old Kernel

Linux Kernel Overview

Where does Linux Kernel belong?

- Linux has a Monolithic Kernel
- Is modular, designed to insert or remove modules while running
 - Loading or removing code on demand
- Is Preemptive
- Supports SMP
- Manages devices through sysfs and device model
- Views threads as processes sharing resources.

Linux Kernel Overview



Linux Kernel Overview

Major blocks of Linux Kernel

Process Management:

- Deals with process.

- Creating and Destroying , Scheduling, IPC

Memory Management:

- Deals with how memory is shared among processes and Kernel.

- Virtual Address Space

Filesystems:

- Everything is file philosophy, VFS.

- Multiple filesystems supported

Device Control:

- Everything other than processor and memory.

- Device drivers

Networking:

- N/w Packet management.

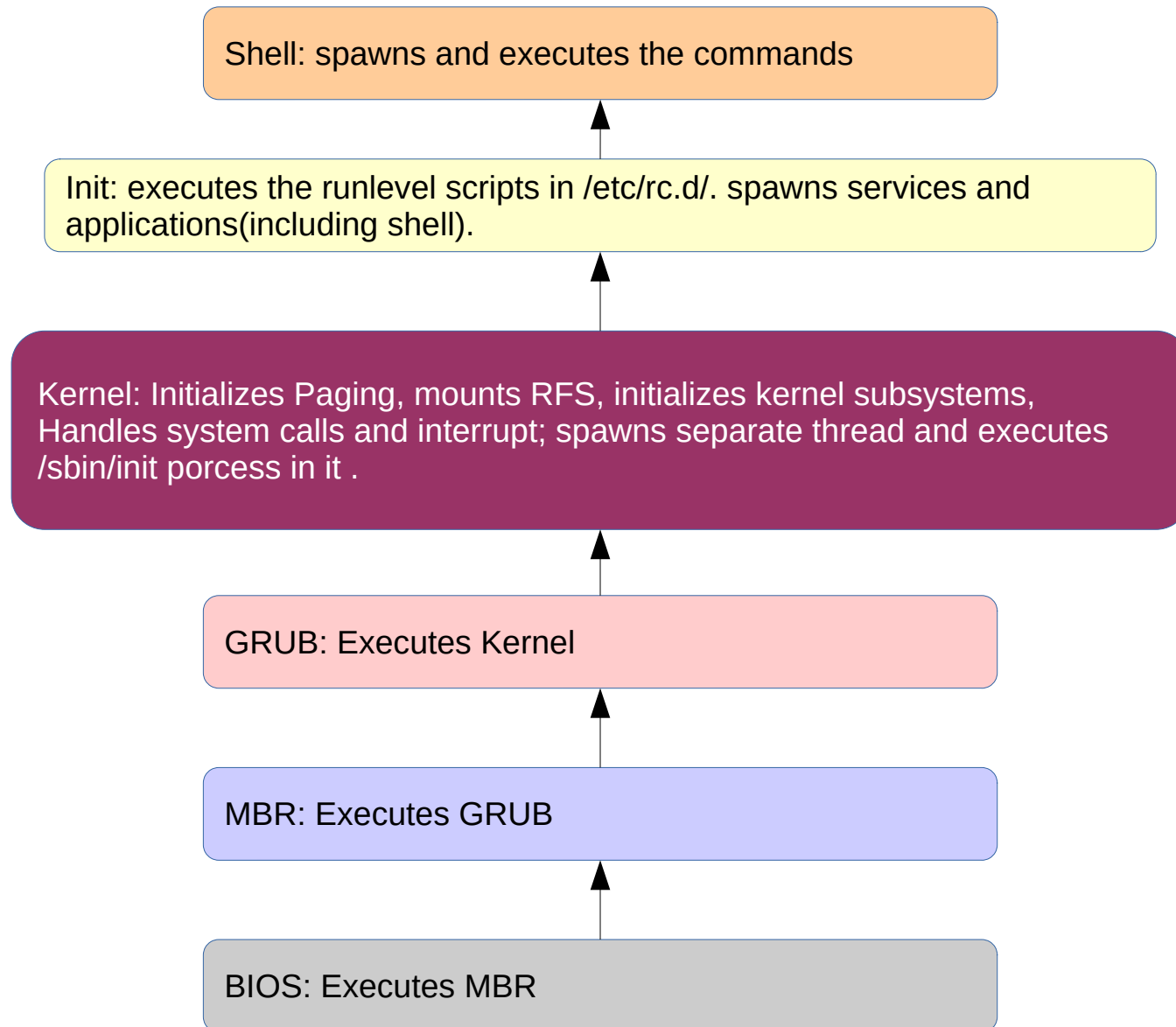
- Implements N/w protocols

Working with Kernel

What's a device driver?

- Software layer between user application and device.
- Hold the complete knowledge of device
- Provide a Mapping of user commands/kernel events to device operations.
- Hide complexity of handling device from Kernel
- Modular drivers are easy to write

Linux Booting Process on a PC



User Mode vs Kernel Mode

H/W Privilege level

Defines the instruction set that can be executed and virtual memory address that can be accessed.

Eg: Intel IA-32 provides 4 privilege levels: rings0-3

- Ring0 is the most privileged mode
- Privilege decreases with increasing ring number.

Linux uses hardware support of these rings(0, 3) to implement

- Privileged kernel mode and
- Non-privileged user mode

User Mode vs Kernel Mode

Processor modes

In Kernel mode the processor:

- Has access to all the instructions
- Whole of the virtual address space.

In User mode the processor:

- Is allowed restricted subset of instructions
- Can access memory from 0 - TASK_SIZE (Eg: In IA-32 it is 0 – 3GiB)

User Mode vs Kernel Mode

Processor modes

Switching from user mode to kernel mode occurs when processor encounters

- Exception
- Faults
- Interrupts

When an exception occurs in User mode:

- User mode execution freezes
- respective exception handler in the Kernel is executed and
- Control switches back to user mode where it left off.

System call

What is a System call?

Function invocations made from user space into the kernel, in-order to request some service or resource from the operating system.

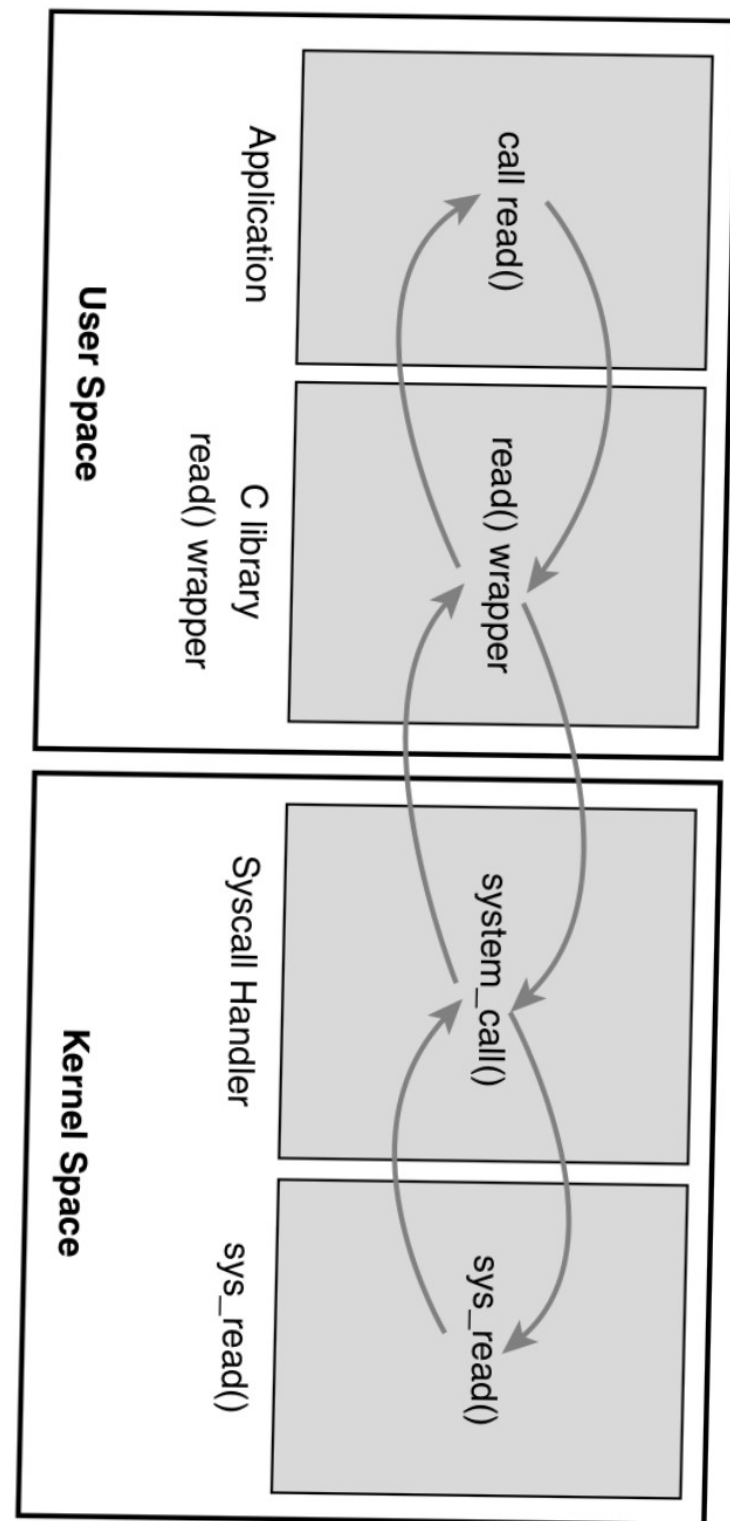
Around 90% of system calls are common to all architecture. In these system calls the programming Interface remain same but are internally implemented according to the architecture.

10% are architecture specific system calls.

System call

Implementation

- System call implementation is architecture specific.
- Each system call is assigned a syscall number.
- The C library encapsulates system call implementation.



System call

Implementation

- When a system call is invoked, application specifies the syscall number.
On i386 eax register is used to convey syscall number.
- Using architecture specific mechanism enter the Kernel space
On i386, a user-space application executes a software interrupt instruction, int, with a value 0x80 (int 0x80). This instruction causes a software interrupt
The control switches into system call interrupt handler routine in Kernel space.
- All system calls enter Kernel space through same mechanism and are then Identified by syscall number.
- Newer architecture support quicker ways of handling system call switching.
Intel x86 has SYS_ENTER and SYS_EXIT
AMD has SYSCALL and SYSRET